


# Efficient GPU Multitasking with Morphable Kernels

Tingxu Ren, Ruwen Fan, Hao Guo, Minhui Xie, Shiwei Gao, Jiwu Shu, Youyou Lu*
Tsinghua University

## Abstract

GPU multitasking offers a promising approach to improving hardware utilization by co-locating concurrent workloads on the same device. However, achieving high resource utilization with minimized interference requires fine-grained, adaptive scheduling. Existing scheduling solutions are fundamentally constrained by a rigid assumption: once a kernel is launched, its resource footprint remains fixed throughout its execution. Consequently, they either rely on static resource pre-partitioning, which lacks the flexibility to adapt to rapid workload changes, or adopt kernel-slicing techniques, which achieve fine-grained control at the cost of prohibitive kernel launch overheads.

In this paper, we break this constraint by introducing *morphable kernels*, a new software abstraction that allows a kernel to dynamically yield or reclaim compute resources on the fly without additional launches. Building on this abstraction, we design MorphX, a GPU multitasking system that embraces the runtime adjustability of kernels as the first principle. MorphX transparently transforms traditional kernels into morphable ones at compile time, and leverages an in-place profiling mechanism to orchestrate fine-grained scheduling based on real-time resource contention. Evaluation under various scenarios shows that MorphX can improve background task throughput by 2.24x under the same foreground task latency, compared to existing systems.

***CCS Concepts:*** • **Computer systems organization → Parallel architectures**.

***Keywords:*** GPU multitasking, GPU scheduling, Resource management

**ACM Reference Format:**
Tingxu Ren, Ruwen Fan, Hao Guo, Minhui Xie, Shiwei Gao, Jiwu Shu, Youyou Lu. 2026. Efficient GPU Multitasking with Morphable Kernels. In *Symposium on Operating Systems Principles (SOSP '26), September 29–October 2, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3830418.3843904

*Youyou Lu (luyouyou@tsinghua.edu.cn) is the corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
*SOSP '26, Prague, Czech Republic*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/26/09
https://doi.org/10.1145/3830418.3843904

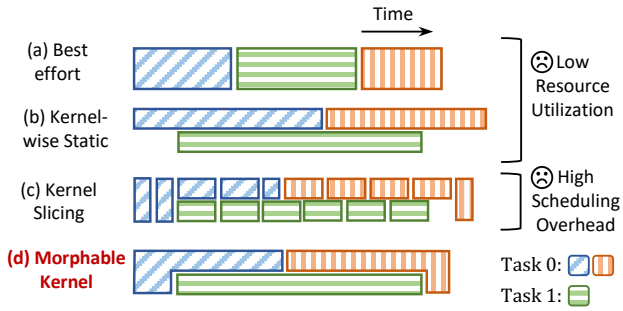



**Figure 1. GPU task scheduling methods.** Best-effort methods cannot explicitly allocate SMs or memory bandwidth, while kernel-wise static methods support only rigid allocation, resulting in low resource utilization. Kernel slicing methods enable fine-grained scheduling, but incur frequent extra kernel launches and thus introduce high scheduling overhead. Morphable kernels adjust SM allocation without extra kernel launches.

## 1 Introduction

GPU-accelerated tasks have consistently delivered high performance across a wide range of domains, including graphics rendering [5, 37], high-performance computing (HPC) [21, 45], and machine learning (ML) [12, 46]. However, GPU resources, particularly Streaming Multiprocessors (SMs) and memory bandwidth, are often underutilized. A single kernel is typically bottlenecked by either compute capacity or memory bandwidth, leaving the complementary resource idle. Such underutilization has also been widely observed in production environments [6, 8, 15, 48].

GPU multitasking [6, 44, 52, 53] offers a highly promising solution to this inefficiency by co-locating compute-intensive and memory-intensive tasks (e.g., the prefill and decode phases of Large Language Models (LLMs) [7, 60], the inference and vector search tasks in Retrieval-Augmented Generation (RAG) [16, 20], etc) for concurrent execution. Yet, achieving high multi-resource utilization while minimizing inter-task interference requires extremely fine-grained, adaptive resource scheduling.

Unfortunately, existing GPU scheduling solutions [4, 6, 24, 31, 32, 44, 52, 57, 58] are fundamentally constrained by a rigid assumption inherited from the conventional GPU execution model: once a kernel is launched, its resource footprint (e.g., the occupied SMs and memory bandwidth) is immutable and remains fixed throughout its entire execution. Though GPU preemption systems [11, 51] enable preempting the SMs of a kernel during its execution, they neglect kernel resource

demands and fail to leverage resource-complementary scheduling, resulting in sub-optimal GPU utilization. Because the scheduler loses control over a kernel's resources once it lands on the hardware, existing systems are forced into a harsh dilemma between scheduling flexibility and execution overhead.

On one side of the dilemma are coarse-grained scheduling approaches [4, 24, 31, 32, 44, 52, 57]. As shown in Figure 1(a) and 1(b), they either rely on CUDA streams for best-effort scheduling [24, 44, 52] or statically allocate SM resources before kernel launches [4, 31, 32, 57]. However, because a task may co-run with different kernels at different points during its execution, a fixed allocation policy often becomes sub-optimal, leading to underutilized resources such as idle SMs or memory bandwidth. To address the inflexibility of static allocation, a second line of work adopts fine-grained scheduling based on kernel slicing techniques [6]. As shown in Figure 1(c), by decomposing the kernel into slices and controlling SM allocation for each kernel slice, they achieve higher utilization of both resources. However, this method incurs significant scheduling and kernel launch overheads since each slice is launched as a separate kernel [25, 61]. Specifically, with a single kernel launch costing upwards of 10 µs, aggressive kernel slicing can increase matrix multiplication latency by 5.317x in our measurements.

To break this dilemma, we introduce *morphable kernels*. Unlike conventional GPU kernels, whose occupied SM resources are fixed once launched and remain unchanged throughout execution, a morphable kernel can dynamically adjust the number of SMs it occupies (yielding or reclaiming part of the SMs) on the fly. This distinction turns the GPU kernel from a rigid computational function into a runtime-controllable scheduling object, thereby creating new opportunities for GPU multitask scheduling. On one hand, it allows the scheduler to apply fine-grained resource allocations according to changing co-location contexts and achieve high resource utilization. On the other hand, it enables fine-grained control without incurring extra kernel launch overheads.

However, realizing and exploiting morphable kernels is non-trivial due to two key challenges. First, how to transform traditional kernels into morphable kernels in a non-intrusive manner. GPUs provide neither an interrupt mechanism nor a control interface for running kernels. This requires an equivalent software mechanism that is correctness-preserving and non-intrusive to existing applications. Second, how to dynamically control morphable kernels to apply the optimal resource-allocation strategy at each moment. The system needs to maintain real-time awareness of the currently running kernels and their resource usage. Yet such information is not directly exposed by the GPU hardware, necessitating a software-based, lightweight alternative.

To overcome the challenges, we design MorphX, a prototype that implements the morphable kernel in a non-intrusive manner and leverages it for GPU multitask scheduling.

First, to transparently transform GPU kernels into morphable kernels, we introduce a compile-time technique. Our key observation is that GPU kernels can be partitioned into minimal schedulable units that can be remapped arbitrarily across SMs without violating program semantics. Based on this observation, we morph the kernel at the granularity of these units. Specifically, a dedicated micro-scheduler is deployed on each SM for each morphable kernel. It continuously fetches and executes kernel units. When the scheduling policy changes, the micro-schedulers can yield or reclaim SMs, allowing arbitrary subsets of SMs to be preempted and resumed on the fly.

Second, to efficiently schedule morphable kernels, we design an in-place resource monitoring mechanism and corresponding fine-grained scheduling strategies. We observe that a kernel's resource demand is largely reflected in its per-unit latency under different SM allocations: for a memory-bandwidth-bound kernel, the per-unit latency increases as more SMs are allocated, whereas for a compute-bound kernel, it remains nearly unchanged. Based on this, MorphX leverages the design of morphable kernels to collect lightweight runtime statistics. It then recognizes real-time resource utilization and automatically derives optimal SM partitioning policies for co-located kernels. This design eliminates the need for offline profiling, thereby improving scalability.

We evaluated MorphX across various scenarios, including co-location of different applications and the disaggregated inference for LLM serving. Experiments show that MorphX achieves the highest multi-resource utilization among all systems. It improves background task throughput by up to 2.24x under the same foreground task latency as the SotA. It also mitigates resource contention and reduces task latency by up to 63.3% under heavy pressure, compared to the SotA. Evaluations on prefill-decode co-located LLM serving show that MorphX reduces P99 latency by up to 70.6% compared to the vLLM serving framework [18].

## 2 Background

### 2.1 GPU Overview

**Programming Model.** A GPU task typically consists of a series of GPU kernels, each implementing a relatively independent and modular operation. Kernels are hierarchically structured, comprising multiple thread blocks, which in turn consist of individual threads [26], as illustrated in Figure 2. Each thread obtains its globally unique index via the `blockIdx` and `threadIdx` registers. Threads within the same block can synchronize through dedicated instructions. Furthermore, modern GPUs usually support streams, e.g., CUDA streams. Within a single stream, kernels execute in a strict sequential order, while kernels from different streams can execute asynchronously and concurrently. However, there is

| Technique | Class | SM Partitioning Unit | Extra Launch | Asynchrony | Resource Complementary | Contention Awareness |
|---|---|---|---|---|---|---|
| Reef [11] | Kernel preemption | None | Low | ✓ | ✗ | C |
| FLEP [51] | Kernel preemption | Kernel | Low | ✓ | ✗ | C |
| TGS [52] | Best effort | None | Low | ✓ | ✗ | C |
| Paella [24] | Best effort | None | Low | ✓ | ✗ | C |
| Orion [44] | Best effort | None | Low | ✓ | ✓ | C |
| Dilu [22] | Best effort | None | Low | ✓ | ✓ | C&M |
| BLESS [56] | Kernel-wise static | Context | Low | ✗ | ✓ | C&M |
| SGDRC [57] | Kernel-wise static | Kernel | Low | ✗ | ✓ | C&M |
| LithOS [6] | Kernel slicing | Kernel | High | ✗ | ✓ | C&M |
| **MorphX** | **Morphable kernel** | **Morph. Kern.** | **Low** | ✓ | ✓ | **C&M** |

**Table 1. Comparison of state-of-the-art GPU multitasking techniques.** *Asynchrony*: Kernels are launched and scheduled without host-side synchronization. *Resource Complementarity*: Kernels with complementary resource characteristics are co-scheduled to improve hardware utilization. *Contention Awareness*: Indicates whether the technique accounts for compute (C) and memory bandwidth (M) contention.

no guarantee how kernels in different streams will be scheduled by the hardware, even when different stream priorities are used [44].

**Hardware Architecture.** On the hardware side, a GPU is composed of numerous Streaming Multiprocessors (SMs); the NVIDIA A100, for instance, incorporates 108 SMs. The high-bandwidth memory (HBM), L2 cache, and PCIe bandwidth are shared among SMs. Each SM possesses its own dedicated compute resources, including the register file, shared memory, and CUDA/tensor cores. When a kernel is launched, its required amount of registers, shared memory, and number of threads are determined. Based on this information, the driver calculates the maximum number of thread blocks that can concurrently reside on one SM. The hardware then distributes the thread block to available SMs that have available resources. As an example, Figure 2 shows a GPU with 4 SMs executing two kernels. The maximum number of thread blocks residing on an SM are 4 and 2 for them, respectively. Once assigned to a particular SM, a thread block cannot be migrated until its execution completes. The hardware scheduler entirely controls the distribution of thread blocks.

### 2.2 GPU Multitasking

**GPU Multitasking Practices.** GPU multitasking has been widely adopted in various domains. Diverse scheduling strategies have been developed to co-locate Deep Neural Network (DNN) workloads to improve device utilization [11, 24, 44]. With the rise of LLM, GPU tasks are showing increasing diversity in resource demands. The prefill and decode phases of LLM are known to have complementary resource requirements. Extensive research has focused on the temporal and spatial multiplexing in LLM serving [7, 8, 60]. Recently, advances in RAG [16, 20] and Reinforcement Learning [36, 41] (RL) have also raised the potential need to co-locate LLM inference workloads with ANNS or training. There's a growing demand for a dynamic, resource-aware technique to efficiently co-locate various tasks.

**GPU Multitasking Techniques.** Table 1 compares representative GPU multitasking techniques by their SM partitioning unit, launch overhead, and resource awareness. Following the classification in the introduction, existing resource-allocation techniques mainly use best-effort scheduling, kernel-wise static allocation, or kernel slicing.

Best-effort methods [22, 24, 44, 52] coordinate kernel launch timing through CUDA streams, while leaving thread-block placement to the hardware scheduler. They require no additional kernel launches, but cannot guarantee a specific SM allocation. Therefore, these methods cannot reliably co-schedule kernels with complementary resource demands and cannot observe runtime contention.

Static methods [4, 31, 32, 56, 57] allocate SMs at the contex or kernel level before kernel launches. Typically, they rely on the CUDA Green Context API [29] or specific driver features to enforce such resource partitioning. The allocation is explicit, but remains fixed while a kernel is running. These methods therefore provide limited flexibility when the co-running kernels or their resource demands change.

Kernel-slicing methods [6] divide a kernel into separately scheduled slices and assign resources to each slice. Slicing provides finer-grained control, but flexible slice-level scheduling requires a separate kernel launch and host-side synchronization for every slice. As shown in Table 1, this leads to higher launch overhead.

Another line of work targets GPU preemption [11, 51]. These methods allow preempting a running kernel for latency-critical scheduling. However, these systems are mostly resource-agnostic: they do not model compute and memory-bandwidth

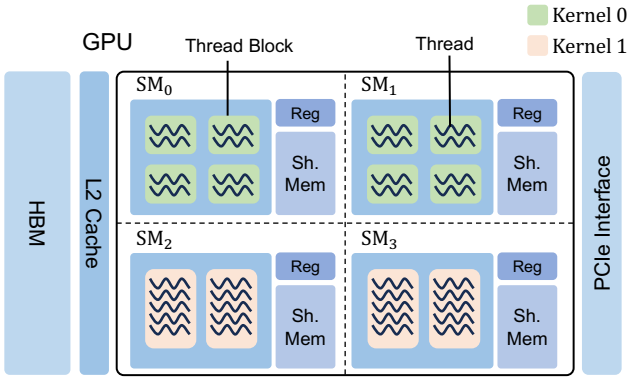



**Figure 2. GPU hardware architecture and programming model.** A GPU device is composed of multiple SMs, each with its own compute units, registers, and shared memory. HBM, L2 cache, and PCIe are shared. Kernel 0 and Kernel 1 occupy $SM_{0,1}$ and $SM_{2,3}$, respectively. Reg: Register file. Sh. Mem: Shared memory.

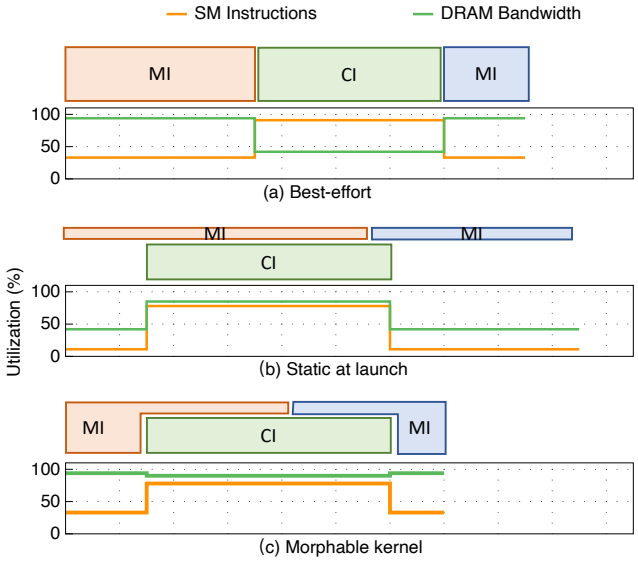



**Figure 3. Resource utilizations when applying different mechanisms**. Morphable kernel shows promise in increasing utilization of both SM and memory bandwidth resources with the ability of on-the-fly adjustment. Bars show execution intervals and curves show resource utilization. (MI: memory-intensive. CI: compute-intensive.)

demands to identify complementary kernels or select allocations that improve multi-resource utilization.

## 3 Motivation

We start by investigating how different methods help improve resource utilization. Figure 3 briefly shows the compute resource and memory bandwidth utilization when applying different approaches. We choose three GEMM operators from CUTLASS [28] that are either memory-intensive or compute-intensive. Utilizations are profiled using NSYS [34]. As morphable kernels are currently not available, we separately profiled each possible SM configuration using the CUDA Green Context [29] and concatenated the results.

According to the profiling results, the best-effort method underutilizes either compute resources or memory bandwidth because it relies on sequential execution. For the kernel-wise static method, prior profiling indicates that the optimal SM partition for co-locating CI and MI kernels is 3 : 1. This configuration achieves high utilization of both resources, thus maximizing throughput. However, MI kernels must remain pinned to this partition due to the inability to modify SM allocation on-the-fly. As a result, when no CI kernel is present, some SMs are left idle, leading to poor overall resource utilization.

In contrast, the morphable-kernel method achieves a near-optimal SM allocation strategy at each moment. Specifically, it can dynamically reduce the number of SMs assigned to the memory-intensive kernel so that it can be co-located with a compute-intensive kernel under the optimized partition ratio. More importantly, as the set of running kernels changes, it promptly adjusts SM allocations to match the new workload composition, ensuring that the optimal scheduling policy is continuously enforced. As a result, it avoids the rigidity of static partitioning and sustains high overall resource efficiency across different execution phases.

The best-effort and kernel-wise static methods fail because a task's dominant resource demand can vary rapidly over time. As a result, the kernel co-located with another kernel may shift between different resource bottlenecks during its lifetime. This phenomenon is common in modern GPU workloads. For example, in LLM serving with relatively large batch sizes ($\geq 128$), the attention operator is typically memory-intensive, whereas the FFN layers are largely compute-intensive. Therefore, execution alternates between these two types of kernels from layer to layer, rendering a static SM allocation strategy inadequate. Moreover, modern large-scale training and inference workloads also involve communication-heavy operators, such as the all-to-all operations used in MoE models [14]. These operators appear repeatedly across layers, yet consume primarily interconnect bandwidth, leaving a substantial fraction of GPU compute capacity underutilized.

This behavior necessitates fine-grained GPU multitask scheduling so that resource allocation can adapt to changing workload characteristics over time. However, such fine-grained scheduling is fundamentally constrained by the conventional GPU execution model. Today's GPU software stack treats a kernel as a monolithic execution unit whose resource

footprint is fixed at launch time. As a result, achieving fine-grained control often requires dividing kernels into small slices, which incurs excessive kernel launch overhead.

Morphable kernels provide a promising way to break this tradeoff. However, realizing this abstraction and leveraging it effectively are non-trivial due to two key challenges.

*Challenge 1: How to enable software-defined kernel morphability.* On-the-fly SM partitioning is difficult because current GPUs expose neither an interrupt mechanism nor a software control interface for running kernels. Once a kernel is launched, its thread blocks are placed onto SMs entirely by hardware, and software can neither control their placement nor reclaim occupied SMs before completion. Although recent techniques [4, 29] support setting a kernel's SM affinity before launch, such allocation remains fixed during execution and thus cannot support dynamic morphing. A practical solution must therefore bypass the best-effort hardware scheduler and expose software-controlled, runtime-adjustable SM management, while remaining transparent and non-intrusive to existing kernels.

*Challenge 2: How to achieve lightweight, real-time resource observability for morphing decisions.* Morphable kernels enable on-the-fly SM reallocation, but exploiting this flexibility requires the scheduler to observe runtime resource usage in real time. This is challenging because current GPUs provide no low-overhead mechanism for online resource tracking. CUPTI [27] is mainly designed for offline profiling, while binary-instrumentation-based approaches [13, 47] incur prohibitive overhead for continuous monitoring. The asynchronous CPU–GPU execution model further complicates the problem as the CPU is not aware of which kernel is currently running. This necessitates a software-based, lightweight mechanism to provide kernel runtime statistics.

## 4 Design and Implementation

To enable fine-grained GPU resource control with low scheduling overhead, we propose *morphable kernels*, i.e., kernels whose occupied SM count can be modified during execution. Based on this abstraction, we design a prototype system MorphX that applies fine-grained low-overhead scheduling.

Figure 4 presents the architecture of MorphX. As a general system, it seamlessly applies to all forms of GPU applications, including ML frameworks such as PyTorch [38], HPC libraries, and user-launched kernels. During compilation, the kernels are automatically transformed into a morphable kernel with control interfaces injected. When applications are executed, common CUDA APIs are intercepted via the `LD_PRELOAD` environment variable. On each kernel launch, the scheduler determines whether to replace the original kernel with the transformed one. On the host side, a metadata queue is maintained for each stream to track the device's execution progress. The scheduler continuously collects kernel runtime information through a set of micro-interfaces. The

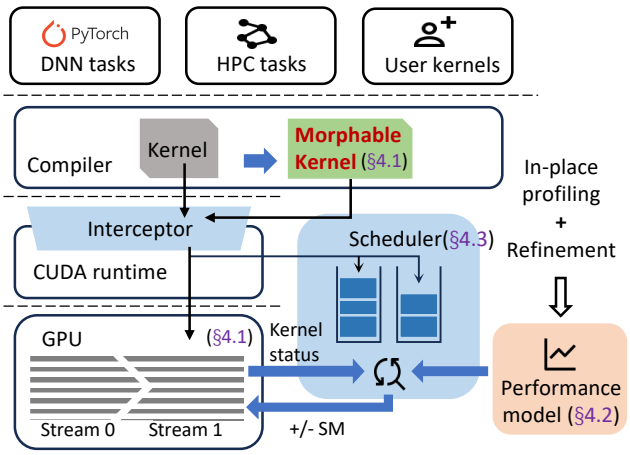



**Figure 4. MorphX architecture overview.** MorphX is non-intrusive and adapts to various GPU applications. The compiler transforms kernels, while the interceptor and scheduler use micro-interfaces and an online model to adjust SM allocation.

information is combined with an interference-aware performance model to infer the kernel's resource demands and recognize contention. Based on this, scheduling decisions are made to adjust the SM allocation of active or pending kernels.

### 4.1 The Morphable Kernels

Traditionally, GPU kernels are executed as monolithic units whose SM allocation is fixed once launched. Morphable kernels transform this execution model by turning kernels into runtime-adjustable execution objects with control interfaces. To realize this abstraction, MorphX applies a compile-time transformation that enables fine-grained, intra-kernel SM placement while remaining transparent to GPU programmers. As a kernel executes, it can dynamically yield parts of the occupied SMs. Once SMs are released, they can be immediately reclaimed by other concurrent tasks.

**Key observation.** We observe that a GPU kernel can be partitioned into a collection of fine-grained schedulable units (i.e., thread blocks) that can be remapped across SMs during execution without affecting correctness. Therefore, MorphX views the thread blocks as a shared work pool and manages their placement on SMs in software.

**Yielding SMs on the fly.** The difficulty of software-controlled SM management lies in the opaque hardware scheduling mechanism of conventional GPU kernels, which hides thread block placement from software once a kernel is launched. To enable morphable kernels, MorphX moves thread block dispatch into software. Specifically, for each kernel, MorphX deploys a lightweight *micro-scheduler* on each SM. These micro-schedulers repeatedly fetch and execute thread blocks from the shared work pool, thereby preserving the original

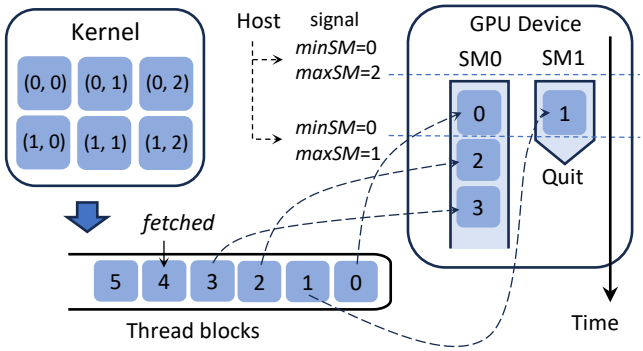



**Figure 5. Yielding SMs on the fly**. *fetched* represents the global index increased by the micro-schedulers to fetch thread blocks. Reducing *maxSM* from 2 to 1 makes SM1 quit after its current block while SM0 continues, releasing SM1 without relaunching the kernel.

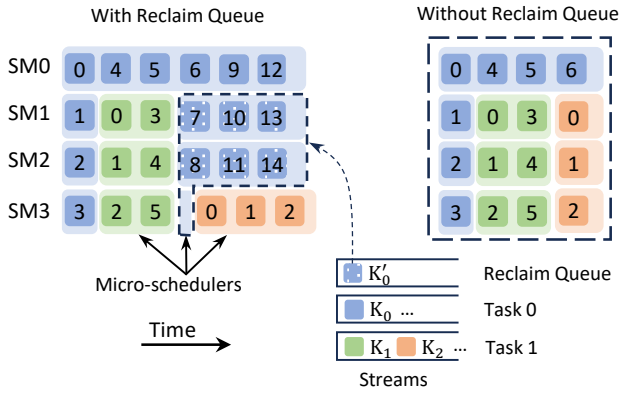



**Figure 6. SM reclaiming**. To reclaim SMs, the same kernel is launched in the reclaim queue with the same *fetched* and *finished* pointers. The high-priority $K_0'$ reuses SMs released by $K_1$ while $K_2$ proceeds, avoiding head-of-line blocking (left); without the queue, $K_2$ waits (right).

kernel semantics. The host-side scheduler governs the active SM range of a kernel through two control signals, *minSM* and *maxSM*. When the scheduling policy changes, micro-schedulers outside the target range stop fetching new thread blocks after completing their current ones, thereby yielding the corresponding SMs for other concurrent kernels.

Figure 5 illustrates the yielding process. One micro-scheduler is deployed on each SM. Each micro-scheduler fetches a new thread block by atomically incrementing a global index *fetched*. The returned index is then converted to the corresponding three-dimensional thread block index in the original kernel. In this example, the kernel initially occupies all SMs, and the micro-schedulers on SM0 and SM1 fetch and execute the first two thread blocks, respectively. The host scheduler then lowers *maxSM* to 1. After completing its current thread block, the micro-scheduler on SM1 stops fetching the next one and relinquishes SM1, which can then be immediately occupied by kernels from other streams. Meanwhile, the micro-scheduler on SM0 continues fetching and executing the remaining thread blocks until the kernel completes.

**Reclaiming yielded SMs.** While the above mechanism enables a running kernel to yield SMs, it does not by itself support reclaiming previously yielded SMs. Relaunching the kernel in its original co-located stream is ineffective because other asynchronously launched kernels in the same stream may block it, causing head-of-line (HoL) blocking. Aborting those pending kernels and relaunching them is possible, but would introduce substantial launch overhead.

To address this problem, MorphX introduces a *reclaim queue*. As shown in Figure 6, MorphX creates an internal opaque stream in its runtime and assigns it a higher priority than any user-created stream. When the scheduler decides to return idle SMs to a kernel ($K_0$) after its co-located kernel ($K_1$) finishes, it launches a new kernel instance, denoted as $K_0'$, into the reclaim queue. $K_0'$ is identical to $K_0$ except for its *minSM* and *maxSM* settings. Importantly, $K_0'$ shares the same *fetched* counter with $K_0$, so its micro-schedulers continue consuming thread blocks from the same logical work pool as if they were part of the original execution. Because the reclaim queue has the highest priority, $K_0'$ can immediately occupy the SMs released by $K_1$, ahead of the subsequent kernel $K_2$ in the stream of $K_1$, thereby mitigating HoL blocking. In this example, we assume that the scheduler wants kernel $K_0$ to occupy SM0 to SM3, thus $K_0'$ immediately yields SM4 to give way to $K_2$.

Although this method incurs additional kernel launches, the total number of launches is bounded by at most twice that of the original execution. This is because reclamation is triggered only when a kernel exits, so the number of extra launches cannot exceed the number of original kernels. In practice, this overhead is much smaller than that of kernel-slicing-based methods, where a single kernel may be divided into tens or even hundreds of slices.

**Compile-time Kernel Transformation.** MorphX requires no modifications to the user code by implementing the transformation of morphable kernels as a compiler plugin. It leverages the Intermediate Representation (IR) [19] during compilation, employing a suite of optimizer-level passes to automatically transform kernels. IR-level transformation is preferred over PTX (a virtual instruction set for GPU) modification for two primary reasons: 1) enhanced generality on multiple backends, and 2) the avoidance of performance degradation arising from manual register management. As shown in Figure 7, the passes mainly perform the following transformations: ❶ Force `__device__` functions to be inlined. ❷ Append new parameters to all `__global__` functions, which also serve as micro-interfaces for host-device

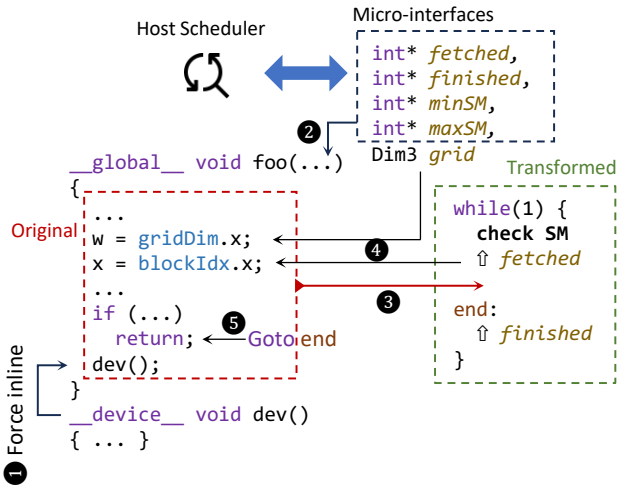



**Figure 7. Compile-time kernel transformation.** The transformation is implemented via an LLVM pass, requiring no manual modification to the code. The numbered steps inline functions, add micro-interfaces, wrap the body in a work-fetching loop, remap indices, and redirect returns.

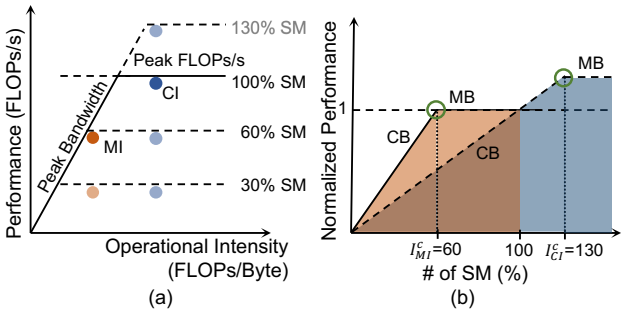



**Figure 8. (a) Original Roofline Model. Reducing SMs lowers the compute ceiling, changing an MI kernel to compute-bound. (b) Performance of different kernels when allocated increasing SMs. The turning point represents the *normalized compute intensity*.** MI: Memory-intensive kernel. CI: Compute-intensive kernel. CB: Compute-bound phase. MB: Memory-bound phase.

communication. The original grid dimensions are passed as parameters because the current grid dimension has become the number of micro-schedulers. ❸ Wrap each kernel with a while loop and insert specific logic. This includes verifying SM ID, updating the hooks, deriving thread block index, etc. For each thread block, this code is executed only by the leading thread to ensure consistency. ❹ Update all occurrences of `blockIdx` and `gridDim`. ❺ Replace all *return* commands with a branch to the end of the while loop. The transformation is independent of the underlying kernel code and introduces negligible overhead.

**Kernel Switching.** Typically, the injected logic introduces little overhead compared to the kernel execution time. However, the overhead becomes non-negligible when the kernel is too small. To solve the problem, we find that it is actually unnecessary to transform these small kernels into morphable kernels. They typically finish quickly and incur little interference to the co-located tasks. Moreover, we find that the dominant kernels in common workloads usually have relatively long latency. Based on this, MorphX applies *kernel switching* when launching the kernel. During compilation, it stores both the original kernel and the transformed kernel in the binary file. On each kernel launch, the scheduler leverages the profiled wave latency to estimate the relative overhead. If it exceeds a certain threshold, the scheduler launches the original kernel instead. In this way, the small, non-dominant kernels remain intact, while large, dominant kernels are enabled for on-the-fly resource partitioning.

**Kernel Micro-interfaces.** The appended parameters are utilized not only for the implementation of morphable kernels but also serve as a micro-interface for host-device control and monitoring. *fetched* and *finished* are atomically incremented by the GPU before and after each thread block. The host scheduler reads them to track kernel execution progress. *minSM* and *maxSM* are written by the host scheduler to control the desired resident SMs for the kernel. The device-side micro-schedulers read them before each thread block starts. Together, they form a lightweight, bidirectional communication interface. This interface occupies only four integer variables, all residing in device memory. The host scheduler accesses them via GPU Direct RDMA Copy (GDRCopy) [30], without involving any GPU compute units. Consequently, the impact on device code execution is minimized. Meanwhile, as shown in §4.2, this micro-interface provides sufficient functionality and flexibility for monitoring GPU execution progress and resource demands.

### 4.2 Real-time Resource Monitoring

To effectively leverage morphable kernels for fine-grained dynamic scheduling, MorphX needs a lightweight yet timely way to monitor runtime behavior. This is challenging because GPU execution is inherently asynchronous, while the hardware exposes no dedicated support for fine-grained real-time resource tracking.

**Key idea.** A key benefit of morphable kernels is that they expose lightweight micro-interfaces to software, making their internal progress partially observable at runtime. MorphX exploits these interfaces to measure per-wave latency online. We observe that a kernel's resource demand is largely manifested in its wave latency, which enables MorphX to infer resource usage in real time.

**Depiction of Resource Demands.** MorphX leverages the high parallelism of GPU kernels to introduce a resource-aware performance model. The model abstracts a kernel's resource characteristics using a unified metric *normalized compute intensity*, and depicts resource contention based on it. We start from the classical Roofline Model [50] (Figure 8(a)), which illustrates how kernels with certain operational intensities are bounded by device resources. *Operational intensity* is defined by the number of computational operations per byte of HBM traffic. For low-operational-intensity kernels (e.g., memory-intensive kernels), performance is bounded by memory bandwidth. In contrast, high-operational-intensity kernels (e.g., compute-intensive kernels) are bounded by the device's compute capability.

Limiting the number of SMs equates to constraining the peak computational performance (FLOPs/s) while maintaining the peak memory bandwidth. By lowering the horizontal line in Figure 8(a), we find that the kernels' behavior can be categorized into two phases: 1) When the ratio of allocated SMs falls below a certain threshold, the kernel is always compute-bound. 2) As the allocated ratio of SMs exceeds the threshold, the kernel becomes memory-bandwidth-bound. For compute-intensive kernels, the threshold may exceed one, making it unattainable. This two-phase behavior is further illustrated in Figure 8(b).

Therefore, we can define the threshold as the *normalized compute intensity* ($I^c$) of a kernel. The relationship of $I^c$ and 1 depicts whether the kernel is memory-intensive or compute-intensive. Furthermore, the performance of a kernel with $x$ proportion of SM $P^*(x)$ can be formulated as:

$$P^*(x) = \begin{cases} \min\left(\frac{x}{I^c}, 1\right) & I^c < 1 \\ x & I^c > 1 \end{cases} \quad (1)$$

Similarly, we can define the *normalized compute intensity* $I^m = \frac{1}{I^c}$ of a kernel, indicating the threshold of supplemented memory bandwidth to saturate compute power. This value helps recognize memory bandwidth contention during kernel co-location. For example, if two co-located kernels ($K_0$, $K_1$) are allocated $x$ and $1-x$ of the SMs, respectively. The desired memory bandwidth to saturate compute power would be

$$\hat{I}^m = xI^m_{K_0} + (1-x)I^m_{K_1} \quad (2)$$

$\hat{I}^m < 1$ indicates under-utilization of memory bandwidth while $\hat{I}^m > 1$ indicates memory bandwidth contention.

**Host-side Kernel Tracking.** To track the lifetime of each kernel, MorphX maintains a metadata queue of the launched kernel for each stream on the host side. When kernels are launched, their metadata, including their micro-interfaces, are pushed to the queue. MorphX does not postpone kernel launches. A global host thread operates in the background to monitor the kernel's execution progress. The thread busy-polls the *fetched* and *finished* interfaces of the kernels at the head of each queue and tracks its progress. When a kernel finishes, the head of the metadata queue is popped.

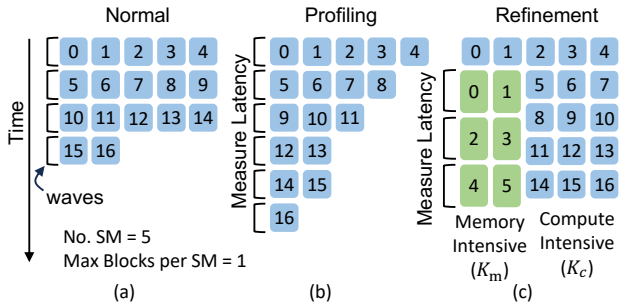



**Figure 9. In-place profiling and refinement.** MorphX measures the per-wave latency to derive the compute intensity of a kernel. (a) Normal waves; (b) profiling with fewer SMs per successive wave; (c) $K_m$ slowdown refines the demand estimate for $K_c$.

By leveraging GDRCopy, the average latency per poll is only about 10 microseconds, much shorter than the typical latency of a single thread block in real-world GPU kernels. A single poll transfers 8 bytes of data, generating a total PCIe bandwidth of about 0.8 MB/s. This is negligible relative to the typical peak bandwidth of about 25 GB/s in modern GPU [42].

**In-place Profiling and Refinement.** MorphX derives a kernel's normalized compute intensity during runtime with negligible overhead. As depicted in Figure 9(a), a kernel typically consists of multiple temporally ordered waves. A wave represents the period from the start to the completion of a thread block. MorphX measures the real-time latency of each wave by recording the time taken for *finished* to increase by the number of currently active micro-schedulers.

A kernel is profiled upon its first execution. As shown in Figure 9(b), a decreasing number of SMs is assigned to successive waves. The total thread block count remains unchanged to guarantee correctness. By recording the latency of each wave, MorphX utilizes Equation 1 to fit the trend of performance changes. Subsequently, it derives the $I^c$ of memory-intensive kernels. The kernel slowdown is at most 2x and is only incurred during the kernel's initial execution. Since a kernel typically executes hundreds of times in a task and there is usually a warm-up phase, the overhead is negligible in most cases.

For compute-intensive kernels, their $I^c$ cannot be derived in a standalone execution. Therefore, the values are temporarily set to 1 and get refined in future co-located executions. As shown in Figure 9(c), when a compute-intensive kernel $K_c$ is co-located with a memory-intensive kernel $K_m$, MorphX measures the latency of the overlapped waves of $K_m$. It then calculates the performance degradation compared with

the standalone profiling results. Since the kernel is memory-bound, the performance degradation should primarily come from memory contention. Suppose that $K_m$ is allocated $x$ of the SM in this execution. Therefore, $I^m_{K_c}$ can be derived by solving the following equation:

$$\frac{\hat{P}_{K_m}(x)}{P^*_{K_m}(x)} = \frac{\min\left(xI^m_{K_m}, 1\right)}{\min\left(xI^m_{K_m}, 1\right) + (1-x)I^m_{K_c}} \tag{3}$$

### 4.3 The MorphX Scheduler

With morphable kernels, MorphX enables more flexible scheduling than traditional schedulers built around static kernels. In this section, we present three scheduling strategies that respectively target resource utilization, performance isolation, and customization. We then use an end-to-end example to show how these strategies interact with runtime monitoring and kernel morphing.

**Throughput-oriented Transparent Scheduling.** By default, MorphX transparently recognizes potential complementary resource demands in concurrent tasks and selects the optimal SM allocation ratio to maximize resource utilization. Specifically, suppose two kernels $p$ and $q$ are co-located, and their normalized memory intensities are $I^m_p$ and $I^m_q$, respectively. We aim to find a division of SM $n_p$ and $n_q$ to fully utilize both the compute resource and the memory bandwidth:

$$\begin{cases} n_p + n_q = 1 \\ n_p I^m_p + n_q I^m_q = 1 \end{cases} \tag{4}$$

Solving this equation yields the optimal SM allocation.

With the help of the real-time monitor, MorphX tracks the $I^m$ of kernels in the front of each host-side queue. However, due to the host-device communication latency, scheduling only the head-of-queue kernel results in at least one wave being unschedulable. Therefore, scheduling decisions are made on the first two kernels of each queue. The second kernel of each stream is checked to see if it has heterogeneous resource demands compared to the kernels in the other stream. If so, SM allocation happens before they are actually executed on the device.

**Memory Bandwidth Isolation.** With the help of its performance model, MorphX can also achieve memory bandwidth isolation to mitigate interference caused by memory contention. According to previous discussions, a kernel's generated memory bandwidth can be limited by reducing its SM count. Specifically, suppose kernel $p$ originates from a high-priority task for which we aim to provide $w^c_p$ of the compute resources and $w^m_p$ of the memory bandwidth. Then the optimal SM allocation to maximize the performance of $q$ can be derived by solving

$$\begin{aligned} \min_{0<n_p\le 1} \quad & n_p \\ \text{s.t.} \quad & n_p \ge w^c_p, \\ & \frac{n_p I^m_p}{n_p I^m_p + \left(1-n_p\right) I^m_q} \ge w^m_p. \end{aligned} \tag{5}$$

**User Interface.** MorphX also provides a flexible interface for manual control of kernel-level SM allocation. For example, in LLM serving, although the prefill and decode stages have different resource demands, using a scheduling strategy that maximizes resource utilization may result in a decreased batch size, thereby failing to achieve the optimal throughput. There are also cases when users hope to set an upper bound on latency for specific tasks. On these occasions, users may provide a hint of the desired percentage of SMs for a stream before launching a kernel. The scheduler will try to satisfy this hint and reduce the number of allocated SMs to the co-located kernels accordingly. If conflicts occur, MorphX prioritizes the later-launched kernels.

**End-to-end example.** Consider two streams: a memory-intensive kernel $K_m$ followed by another dominant kernel $K_n$ in one stream, and a compute-intensive kernel $K_c$ in the other. MorphX transforms these kernels at compile time. At runtime, their asynchronous launches return before GPU execution completes, so the host queues may contain both running and pending kernels; *fetched*/*finished* reveal their actual progress. When $K_c$ is about to execute alongside $K_m$, the scheduler uses their profiled resource demands to select an SM partition, and $K_m$ yields the SMs assigned to $K_c$. While they run, the scheduler also examines the pending $K_n$ and computes the next partition in advance. Suppose $K_n$ needs fewer SMs than $K_m$. Once $K_m$ finishes, $K_n$ starts on its smaller range, while a high-priority reclaim instance $K'_c$, sharing $K_c$'s work-pool counters, occupies the additional released SMs. Thus, MorphX adapts across asynchronous kernel transitions with the optimal SM allocations from its policy.

## 5 Evaluation

We evaluate MorphX to answer the following questions:

- How does MorphX improve resource utilization and mitigate interference in co-location of multiple tasks, e.g., co-locating LLM serving with vector search and matrix multiplication tasks? (§5.2)
- How does MorphX perform in LLM serving with prefill-decode co-location? (§5.3)
- What is the accuracy and overhead of resource monitoring in MorphX? (§5.4)

### 5.1 Experimental Setup

Experiments are conducted on a system with two Intel Xeon Gold 5420+ CPUs, 256GB DRAM, and an NVIDIA A100-40GB GPU. For LLM serving, we adopt PyTorch [38] 2.6.0, vLLM [18] 0.8.1, and FlashInfer [54] 0.2.2 as the backend. Workloads and frameworks are compiled using Clang [19] 21.0.0. Since the cuBLAS used in PyTorch is closed-source, we replace it with the open-source CUTLASS [28], using the optimal operators from our profiling results.

### 5.2 Multi-task Co-location

**Workloads.** In this set of experiments, an LLM serving task is co-located with various background best-effort (BE) tasks. For the LLM serving task, we used Llama-3-8B [2] served with vLLM. The requests are constructed according to ShareGPT [1], a dataset with conversations collected from the ShareGPT API, and follow a Poisson distribution.

We include two workloads as the BE task. The first comprises a GPU implementation of approximate nearest neighbour search [10] (ANNS), which simulates potential use cases in the context of RAG. The second workload comprises a matrix multiplication (MM) task, which is common in Reinforcement Learning [36, 41] and GPU multitenancy in cloud environments. We progressively increase the request rate for BE tasks to simulate varying levels of pressure and interference on LLM tasks. The performance of LLM tasks is measured by 99th percentile (P99) time per output token (TPOT), and the performance of BE tasks is measured by throughput (number of finished operations per second).

**Baselines.** We compare MorphX with the following baselines: ❶ MPS(3:7), where we treat LLM inference as a memory-bound task and adopt the optimized 3:7 ratio in MPS. ❷ MPS(1:1), where we use 1:1 ratio in MPS to ensure fairness. ❸ Stream, where the tasks are launched in two different CUDA streams within the same CUDA context. ❹ Timeslice, where the tasks are launched in two distinct processes and CUDA contexts. Tasks in different contexts are scheduled by the hardware using a time-slice round-robin approach [9]. ❺ TGS [52], a GPU sharing system beneath containers. ❻ Orion [44], a state-of-the-art interference-aware GPU sharing system. ❼ LithOS [6], a representative GPU multitasking system based on kernel slicing. We re-implemented Orion in our codebase to support LLM task scheduling. Since LithOS is closed-source, we also implemented its design in our framework based on its paper.

**Background Workload Throughput vs LLM P99 Latency**. Figure 10 shows the relationship between ANNS throughput and the LLM per-token P99 latencies, under varying pressure. The upper-left represents a higher overall performance and resource utilization. Results show that MorphX constantly outperforms all the baselines. Under the same LLM latency, it improves the ANNS throughput by up to 2.24x compared to the SotA baseline, Orion. This

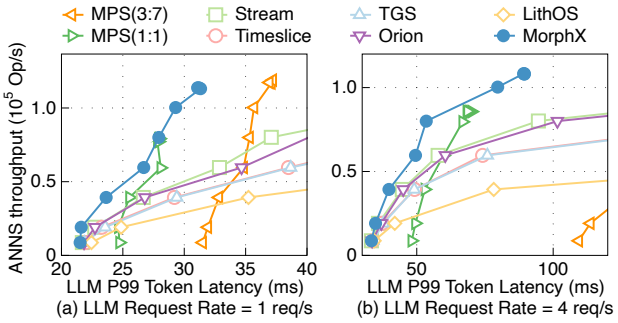



**Figure 10. ANNS throughput vs LLM P99 TPOT.** Upper-left is better.

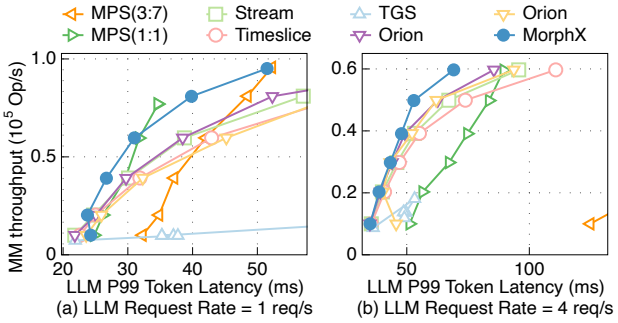



**Figure 11. MM throughput vs LLM P99 TPOT.** Upper-left is better.

is attributed to its ability to adaptively maximize various resource utilization and to control contention.

The baselines fail because they cannot apply resource-complementary scheduling. Neither Stream nor Timeslice supports spatial multiplexing for the two tasks. Therefore, either the compute resource or the memory bandwidth is wasted, leading to reduced overall throughput. Moreover, their uncontrollable temporal multiplexing leads to severe contention when the pressure from ANNS tasks increases. MPS provides a relatively stable inter-task resource partition. However, MPS(3:7) fails to utilize idle SMs under low ANNS pressure, resulting in increased LLM latencies at all times. MPS(1:1) performs relatively well in controlling LLM latencies, yet it fails to adopt an optimal SM partition, resulting in poor ANNS throughput. The SotA baselines, TGS and Orion, also perform poorly because they lack a concrete mechanism for SM partition. LLM and ANNS both feature large kernels, which occupy all SMs and run for a long time. Failing to adjust the SM partition leads to reduced resource utilization and severe interference. LithOS atomizes these long-running kernels and explicitly assigns SMs to each atom. Compared with the coarse-grained baselines, this creates more scheduling points and better isolates the latency-critical LLM from ANNS interference. However, the added flexibility trades

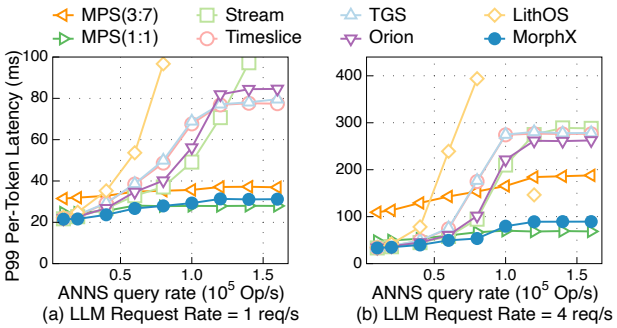



**Figure 12. LLM P99 latency under different background task pressure.** Lower is better. This shows MorphX's efficiency in interference mitigation. Panels use LLM rates of 1 and 4 req/s, with ANNS pressure increasing along the x-axis. MorphX keeps tail latency stable as pressure rises.

off against execution efficiency: every atom requires a separate kernel launch and host-side synchronization, reducing the ANNS throughput that can be achieved at a given LLM latency. In contrast, MorphX changes the allocation of a running kernel without relaunching it, retaining fine-grained control while avoiding these per-atom costs.

Figure 11 changes the background workload to MM and shows its throughput under different LLM latencies. In most cases, MorphX outperforms all baselines. At one point, MPS(1:1) achieves a higher throughput than MorphX. This is attributed to reserving sufficient SMs for LLM. However, this comes at the cost of reduced maximum MM throughput (the highest point in the figure). Orion performs relatively well compared to other baselines. However, as the background pressure increases, the gap between it and MorphX becomes increasingly evident. This suggests its inefficiency in handling high pressure. MPS(1:1) and MPS(3:7) also exhibit commendable performance under a low LLM request rate. However, as the LLM request rate increases to 4 req/s, their performance degrades quickly due to their inability to handle increasing contention.

**Interference mitigation.** Figure 12 shows MorphX's capability to recognize resource contention and mitigate interference. It illustrates the trends in LLM latencies under varying ANNS query rates. With the increase in ANNS query rate, the P99 per-token latency of stream and timeslice grows rapidly and becomes unbearable. MPS(3:7) and MPS(1:1) exhibit relatively stable latency across the LLM task. However, MPS(3:7) exhibits significantly higher latencies across all ANNS request rates. When LLM RPS is 4, it incurs 2.75x to 3.53x longer latencies than MorphX. TGS and Orion also fail in controlling LLM's tail latency. Though adopting certain techniques to mitigate interference, their inability to preempt occupied SM resources in the ANNS kernels results in a constrained scheduling space. LithOS likewise keeps LLM latency competitive at light ANNS load because atomization

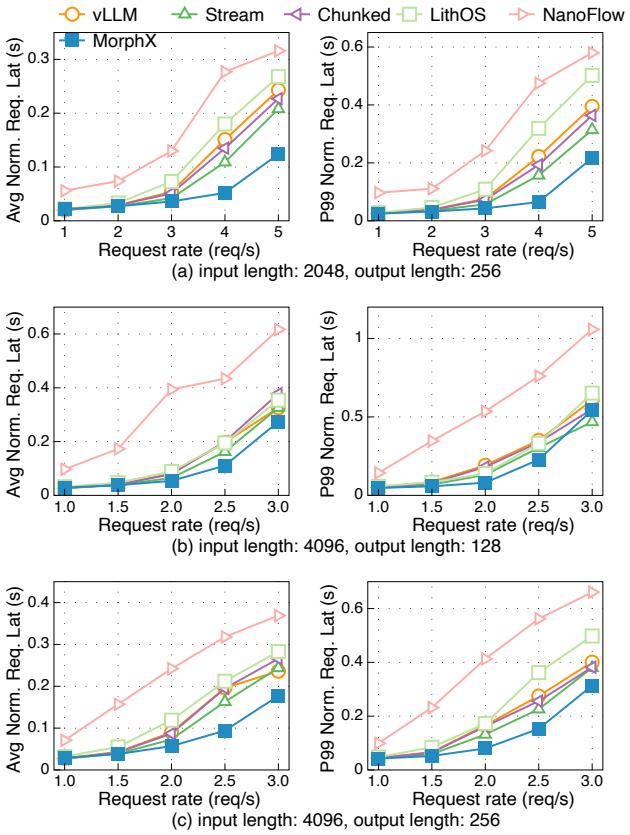



**Figure 13. Qwen 2.5 7B average & P99 request latency.** Lower is better.

prevents one long-running ANNS kernel from monopolizing the GPU. As the ANNS rate increases, however, more atoms must be scheduled, launched, and synchronized; these fixed per-atom costs accumulate, and resource allocation can only be revised at atom boundaries. Consequently, LithOS's tail latency rises sharply and can become worse than even the statically partitioned MPS baselines under heavy load. MorphX instead yields and reclaims SMs within already-running kernels, so it responds to contention without introducing additional launches and maintains stable tail latency as the background pressure grows.

### 5.3 LLM Serving

**Workloads.** In this scenario, we demonstrate the efficiency of MorphX in LLM serving by co-locating the prefill and decode stages. Following the prefill-decode disaggregation technique [59], we modified vLLM to separate the prefill and decode stages into distinct streams and remove dependencies between them. This enables serving the two stages as separate tasks without the overhead of inter-process communication or the duplication of model parameters and KV cache. We adopt two different models: Llama-3-8B and Qwen2.5-7B [40], and use various combinations of input and output

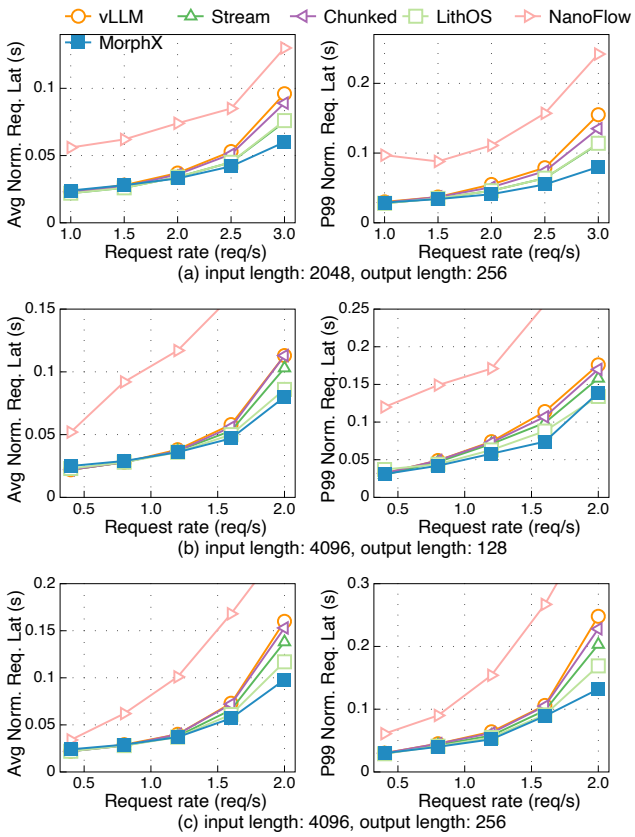



**Figure 14. Llama 3 8B average & P99 request latency.** Lower is better.

lengths. Requests arrive at different rates following a Poisson distribution. Since each setup fixes the input and target output lengths, we report end-to-end request latency normalized by the number of generated output tokens, including both average and P99 values over the evaluation period.

**Baselines.** We compared MorphX with five systems: ❶ vLLM, which represents the default continuous batching scheduling strategy used in vLLM. This strategy uses a single stream and prioritizes prefill to maximize decode batch size. ❷ Stream, which is a modified version of vLLM. Prefill and decode are placed in separate streams and scheduled by the hardware. ❸ Chunked, which stands for chunked prefill [3], a widely adopted technique to batch prefill and decode in a single kernel. It potentially increases resource utilization. ❹ LithOS, as discussed above. ❺ NanoFlow, which is an LLM serving framework that achieves intra-device parallelism by overlapping computation, memory access, and communication within a single GPU.

Figure 13 shows the average (left) and P99 (right) request latency using different scheduling strategies under various context lengths. Various request rates are tested to show the scheduler's efficiency under different pressures. Performance is measured by the latency of each request, defined as the time from the request's arrival to the generation of the last token. With the increase in request rate, MorphX becomes increasingly favorable and dominates the high-contention region. To be specific, under the three different context lengths, MorphX reduces the average request latency by up to 65.4%, 43.7%, and 52.0% compared to vLLM, respectively. Compared to the stream baseline, it also reduces the average latency by up to 51.8%, 31.5%, and 42.1%, respectively. When request rates are low, optimization is not significant because the prefill and decode phases of different requests rarely overlap. However, as the request rate increases, the likelihood of overlapping prefill and decode increases. Therefore, MorphX reduces the average latency by improving resource utilization. The stream baseline also reduces request latency compared to vLLM by leveraging idle SMs. However, it does not fully utilize both compute and memory resources, leaving significant room for optimization. MorphX outperforms stream as it adopts resource-complementary scheduling.

LithOS is competitive at light load, but its slice-level launches and synchronization become costly as request rate rises; MorphX lowers average latency by up to 71.1% and P99 latency by up to 79.6% relative to LithOS in the Qwen workload, and still improves average/P99 latency by up to 21.1%/29.8% in the Llama workload. NanoFlow was slower for two main reasons. First, the workload changed the decode and prefill batch sizes frequently. For each batch size change, NanoFlow had to update its execution plan, buffers, and dependencies during runtime. These updates added CPU and memory-management overhead. Second, only a limited number of operators used colocated execution, so the performance benefit of NanoFlow's scheduling was smaller than its runtime overhead. In essence, NanoFlow is a throughput-oriented system optimized for specific configurations, rendering it ill-suited for our dynamic scenarios.

Nevertheless, MorphX achieves the minimal P99 request latency among all baselines. Across the three context lengths, it reduces the P99 latency of vLLM by up to 70.6%, 58.0%, and 52.1%, respectively. Compared to Stream, it also reduces the P99 latency by up to 58%, 39.1%, and 38.9%, respectively.

Figure 14 shows the average and P99 request latency for Llama 3 8B. Similarly, MorphX achieves the lowest latency as the request rate increases, reducing HoL blocking. To be specific, under the three different context lengths, MorphX reduces the average request latency by up to 37.5%, 29.1%, and 38.9% compared to vLLM, respectively. It also reduces the P99 latency by up to 48.4%, 36.1%, and 47.0%, respectively. In summary, MorphX consistently achieves high resource utilization and performance improvements across different model types and context lengths.

### 5.4 Technical Analysis

**Kernel Statistics.** In the Llama-3-8B serving workload in Figure 14, MorphX observes 26 distinct kernel implementations. Five have a first-wave latency above the 50 µs threshold

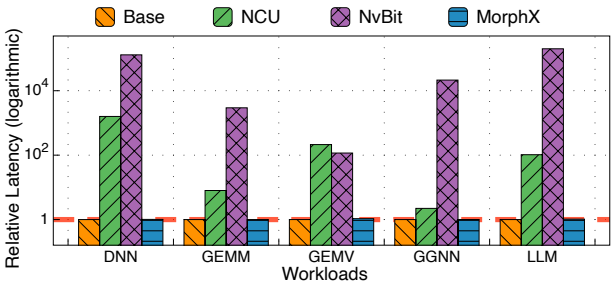



**Figure 15. Overhead of different resource monitoring methods.** The latency is normalized and shown on a logarithmic scale. Base is 1×; lower is better. NCU and NvBit incur orders-of-magnitude overhead, whereas MorphX adds less than 5%.

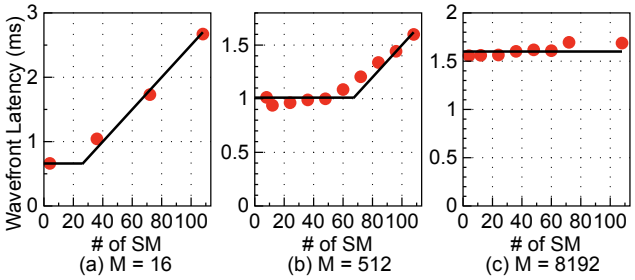



**Figure 16. Wave latency vs Number of allocated SMs.** This figure shows the JIT profile results (§4.2) of different kernels. Kernels are matrix-matrix multiplication with $N = K = 65536$ and varying $M$. Markers are measurements and lines are model fits; (a)–(b) show memory-intensive turning points, while (c) shows a compute-intensive kernel.

and together account for 85% of the total GPU execution time; three are memory-intensive and two are compute-intensive. MorphX enables morphing and monitoring only for these five dominant kernels, while the other 21 execute as their original kernels. This selective instrumentation covers most execution time while avoiding overhead on short kernels.

**Overhead in Real-time Monitoring.** Figure 15 shows the overhead of various resource monitoring methods. NCU leverages the CUPTI library [27, 33] to read hardware counters and gather device information for each kernel. In this experiment, we only collect two metrics, e.g., memory read/write sizes, to minimize overhead. However, NCU still incurs 2.23x to 1590x overhead under the workloads. NvBit is a SASS-level instrumentation tool. We leverage it to track each memory access and further derive memory bandwidth utilization. This method incurs an overhead of more than 409x. Both techniques are inapplicable to real-time monitoring due to massive overhead. In contrast, the overhead of MorphX is less than 5% for all the workloads. For GEMM and GGNN, the overhead is less than 0.5%. This is attributed to the lightweight design of MorphX's kernel micro-interface.

There are also other methods for resource awareness, including NSYS [34] and NVML. However, NSYS only supports offline profiling and provides no runtime access interfaces. NVML is used for system-wide device management. It only includes brief information about the overall utilization (by all tasks) and employs a millisecond-level low-frequency sampling. This information is far from sufficient for on-the-fly scheduling. Therefore, these methods are also unsuitable for real-time monitoring.

**Performance Model Accuracy.** Figure 16 shows the accuracy of MorphX's performance model and the JIT profiling mechanism. It illustrates the relationship between the wave latency and the number of allocated SMs. The kernels in (a) and (b) are memory-intensive. Initially, as the number of allocated SMs increases, their wave latency remains constant. When the allocated SMs exceed a threshold, their wave latency increases proportionally with the number of SMs. The kernel in (c) is compute-intensive. Its wave latency remains the same under all SM numbers. This result is largely consistent with the conclusions in §4.2.

MorphX further derives the *normalized compute intensity* ($I^c$) of these kernels. In this example, the $I^c$ of kernels in (a) and (b) are 0.247 and 0.753, respectively. For the third kernel, MorphX calculates its $I^c$ through co-location with a memory-intensive kernel. By co-locating kernel (c) with kernel (a) and an SM partition ratio of 1:1, the wave latency of (a) increases by 43%. Therefore, according to Equation 3, the $I^c$ of kernel (c) is 1.19.

## 6 Discussion

**Assumptions and Applicability** Since MorphX relies on compile-time transformations, it cannot be directly applied to closed-source kernels. However, such closed-source kernels (e.g., cuBLAS) often have high-performance open-source counterparts (e.g., CUTLASS) that deliver comparable performance. MorphX assumes that target workloads meet two key conditions: (1) dominant kernels exhibit relatively long execution latencies, and (2) each kernel is executed repeatedly. Workloads in LLM, ANNS, and graph computing typically satisfy these assumptions. In contrast, small DNN models, which often feature short-lived kernels, may violate the first condition and are thus less suited for MorphX.

Currently, MorphX does not manage HBM capacity or memory I/O. HBM capacity management and swapping represent an orthogonal line of research [18, 35, 43]. Supporting I/O-bound kernels is a promising direction for future work, as these kernels also require only a few SMs to saturate memory bandwidth. While MorphX's transformation and profiling methodologies can be applied to I/O kernels, its performance model and scheduling policies must be adapted to account for I/O-specific behaviors.

**Multi-processing.** Currently, MorphX launches tasks in separate CUDA streams and uses a background thread to

coordinate resource partitioning between them. However, this can be easily extended to tasks in different processes or containers through API remoting [39, 55]. Certain GPU driver APIs can be intercepted and remoted to a background process that manages one or more streams per process and applies all GPU operations within its own GPU context. In this way, kernels from different processes can execute in parallel as if in separate streams. Techniques for GPU API remoting are well established and orthogonal to this work.

**Scalability Across GPU Types.** MorphX is implemented and evaluated on NVIDIA GPUs. However, it can be seamlessly migrated to any GPU type that comprises multiple SMs (or Compute Units) and supports stream-like semantics. MorphX is fully software-based and doesn't rely on most hardware features. Since it's mostly implemented in the LLVM compiler, such migration requires only minimal additional code. For example, AMD GPUs already use LLVM as their compiler. It's only necessary to modify the API names in the interceptor to migrate MorphX to AMD GPUs.

## 7 Related Works

**Kernel Fusion.** Another line of research improves resource utilization through kernel fusion [3, 17, 49, 61]. For example, Chunked Prefill [3] batches the FFN computations of LLM prefill and decode phases, while Pod-Attention [17] further designs specialized kernels to jointly batch prefill and decode attention operators. Other studies exploit instruction-level scheduling or compilation techniques to fuse compute- and memory-intensive operators [23, 61]. Although these approaches can achieve fine-grained optimization, they are typically tailored to specific workloads and primarily exploit scheduling opportunities *within* a single task at compile time. In contrast, our work introduces morphable kernels as a general runtime abstraction for multitasking, enabling dynamic resource repartitioning across co-located kernels without requiring workload-specific fused kernels.

## 8 Conclusion

This paper introduces *morphable kernels*, a new kernel abstraction that enables GPU kernels to modify the number of SMs they occupy on the fly. Morphable kernels expose a new opportunity for fine-grained, low-overhead GPU multitask scheduling. We build MorphX, a prototype scheduling system that realizes morphable kernels in a non-intrusive manner. It leverages lightweight runtime control and in-place profiling for efficient multitasking scheduling. Experimental results across diverse workloads show that morphable kernels can effectively improve GPU multi-resource utilization, mitigate contention, and deliver substantial gains in both throughput and latency.

## 9 Acknowledgements

This work is supported by the New Generation Artificial Intelligence-National Science and Technology Major Project (Grant No. 2025ZD0122101), the National Natural Science Foundation of China (Grant No. 62332011), and the High Performance Computing Center of Tsinghua University.

## References

[1] 2023. ShareGPT. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered.

[2] 2024. Meta-Llama-3-8B. https://huggingface.co/meta-llama/Meta-Llama-3-8B.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 117–134.

[4] Joshua Bakita and James H. Anderson. 2023. Hardware Compute Partitioning on NVIDIA GPUs. In *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 54–66.

[5] Pieterjan Bartels and Takahiro Harada. 2022. Combining GPU Tracing Methods within a Single Ray Query. In *ACM SIGGRAPH Asia Technical Communications*. 17:1–17:4.

[6] Patrick H. Coppock, Brian Zhang, Eliot H. Solomon, Vasilis Kypriotis, Leon Yang, Bikash Sharma, Dan Schatzberg, Todd C. Mowry, and Dimitrios Skarlatos. 2025. LithOS: An Operating System for Efficient Machine Learning on GPUs. In *ACM Symposium on Operating Systems Principles (SOSP)*. 1–17.

[7] Weihao Cui, Yukang Chen, Han Zhao, Ziyi Xu, Quan Chen, Xusheng Chen, Yangjie Zhou, Shixuan Sun, and Minyi Guo. 2025. Optimizing SLO-oriented LLM Serving with PD-Multiplexing. arXiv preprint arXiv:2504.14489.

[8] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving. In *International Conference on Machine Learning (ICML)*. 11905–11917.

[9] Ruwen Fan, Tingxu Ren, Minhui Xie, Shiwei Gao, Jiwu Shu, and Youyou Lu. 2025. GPREEMPT: GPU Preemptive Scheduling Made General and Efficient. In *USENIX Annual Technical Conference (ATC)*. 263–272.

[10] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik P. A. Lensch. 2023. GGNN: Graph-Based GPU Nearest Neighbor Search. *IEEE Transactions on Big Data* 9, 1 (2023), 267–279.

[11] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 539–558.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778.

[13] Songlin Huang and Chenshu Wu. 2025. Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 331–355.

[14] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive Mixtures of Local Experts. *Neural Computation* 3, 1 (1991), 79–87.

[15] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *USENIX Annual Technical Conference (ATC)*. 947–960.

[16] Wenqi Jiang, Suvinay Subramanian, Cat Graves, Gustavo Alonso, Amir Yazdanbakhsh, and Vidushi Dadu. 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*. 974–989.

[17] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 897–912.

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*. 611–626.

[19] Chris Lattner and Vikram Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization (CGO)*. 75–86.

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.

[21] Ben Lund and Justin W Smith. 2010. A Multi-Stage CUDA Kernel for Floyd-Warshall. arXiv preprint arXiv:1001.4108.

[22] Cunchi Lv, Xiao Shi, Zhengyu Lei, Jinyue Huang, Wenting Tan, Xiaohui Zheng, and Xiaofang Zhao. 2025. Dilu: Enabling GPU Resourcing-on-Demand for Serverless DL Serving via Introspective Elasticity. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 311–325.

[23] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 881–897.

[24] Kelvin K. W. Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency Model Serving with Software-defined GPU Scheduling. In *ACM Symposium on Operating Systems Principles (SOSP)*. 595–610.

[25] NVIDIA Corporation. 2019. Getting Started with CUDA Graphs. https://developer.nvidia.com/blog/cuda-graphs/.

[26] NVIDIA Corporation. 2025. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide.

[27] NVIDIA Corporation. 2025. CUPTI. https://docs.nvidia.com/cupti/.

[28] NVIDIA Corporation. 2025. CUTLASS 3.9.1. https://github.com/NVIDIA/cutlass.

[29] NVIDIA Corporation. 2025. Green Contexts. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__GREEN__CONTEXTS.html.

[30] NVIDIA Corporation. 2025. Magnum IO GDRCopy. https://developer.nvidia.com/gdrcopy.

[31] NVIDIA Corporation. 2025. MIG User Guide. https://docs.nvidia.com/datacenter/tesla/mig-user-guide/.

[32] NVIDIA Corporation. 2025. Multi-Process Service. https://docs.nvidia.com/deploy/mps/index.html.

[33] NVIDIA Corporation. 2025. NVIDIA Nsight Compute. https://docs.nvidia.com/nsight-compute/NsightComputeCli/.

[34] NVIDIA Corporation. 2025. NVIDIA Nsight Systems. https://docs.nvidia.com/nsight-systems/UserGuide/index.html.

[35] NVIDIA Corporation. 2026. CUDA Unified Memory. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/unified-memory.html.

[36] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 27730–27744.

[37] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David P. Luebke, David K. McAllister, Morgan McGuire, R. Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: a general purpose ray tracing engine. *ACM Trans. Graph.* 29, 4 (2010), 66:1–66:13.

[38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32. 8024–8035.

[39] Manos Pavlidakis, Stelios Mavridis, Antony Chazapis, Giorgos Vasiliadis, and Angelos Bilas. 2022. Arax: a runtime framework for decoupling applications from heterogeneous accelerators. In *ACM Symposium on Cloud Computing (SoCC)*. 1–15.

[40] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115.

[41] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. 53728–53741.

[42] Zhenghang Ren, Yuxuan Li, Zilong Wang, Xinyang Huang, Wenxue Li, Kaiqiang Xu, Xudong Liao, Yijun Sun, Bowen Liu, Han Tian, Junxue Zhang, Mingfei Wang, Zhizhen Zhong, Guyue Liu, Ying Zhang, and Kai Chen. 2025. Enabling Efficient GPU Communication over Multiple NICs with FuseLink. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 91–108.

[43] Weihang Shen, Yinqiu Chen, Rong Chen, and Haibo Chen. 2026. Towards Fully-fledged GPU Multitasking via Proactive Memory Scheduling. arXiv preprint arXiv:2512.24637.

[44] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *European Conference on Computer Systems (EuroSys)*. 1075–1092.

[45] Aidan P. Thompson, H. Metin Aktulga, Richard Berger, Dan S. Bolintineanu, W. Michael Brown, Paul S. Crozier, Pieter J. in 't Veld, Axel Kohlmeyer, Stan G. Moore, Trung Dac Nguyen, Ray Shan, Mark J. Stevens, Julien Tranchida, Christian Trott, and Steven J. Plimpton. 2022. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications* 271 (2022), 108171.

[46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*. 6000–6010.

[47] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. 2019. NVBit: A Dynamic Binary Instrumentation Framework for NVIDIA GPUs. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 372–383.

[48] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang

Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 5831–5841.

[49] Yuke Wang, Boyuan Feng, Zheng Wang, Tong Geng, Kevin Barker, Ang Li, and Yufei Ding. 2023. MGG: Accelerating Graph Neural Networks with Fine-Grained Intra-Kernel Communication-Computation Pipelining on Multi-GPU Platforms. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 779–795.

[50] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[51] Bo Wu, Xu Liu, Xiaobo Zhou, and Changjun Jiang. 2017. FLEP: Enabling Flexible and Efficient Preemption on GPUs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 483–496.

[52] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. 2023. Transparent GPU Sharing in Container Clouds for Deep Learning Workloads. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 69–85.

[53] Jiarong Xing, Yifan Qiao, Simon Mo, Xingqi Cui, Gur-Eyal Sela, Yang Zhou, Joseph Gonzalez, and Ion Stoica. 2025. Towards Efficient and Practical GPU Multitasking in the Era of LLM. arXiv preprint arXiv:2508.08448.

[54] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv preprint arXiv:2501.01005.

[55] Hangchen Yu, Arthur Michener Peters, Amogh Akshintala, and Christopher J. Rossbach. 2020. AvA: Accelerated Virtualization of Accelerators. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 807–825.

[56] Shulai Zhang, Quan Chen, Weihao Cui, Han Zhao, Chunyu Xue, Zhen Zheng, Wei Lin, and Minyi Guo. 2025. Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing. In *European Conference on Computer Systems (EuroSys)*. 573–588.

[57] Yongkang Zhang, Haoxuan Yu, Chenxia Han, Cheng Wang, Baotong Lu, Yunzhe Li, Zhifeng Jiang, Yang Li, Xiaowen Chu, and Huaicheng Li. 2025. SGDRC: Software-Defined Dynamic Resource Control for Concurrent DNN Inference on NVIDIA GPUs. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 267–281.

[58] Wei Zhao, Anand Jayarajan, and Gennady Pekhimenko. 2025. Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1052–1068.

[59] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 193–210.

[60] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 749–765.

[61] Donglin Zhuang, Zhen Zheng, Haojun Xia, Xiafei Qiu, Junjie Bai, Wei Lin, and Shuaiwen Leon Song. 2024. MonoNN: Enabling a New Monolithic Optimization Space for Neural Network Inference Tasks on Modern GPU-Centric Architectures. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 989–1005.